

CAP-EVAL-001

프론티어 모델 교차 실험 판정 보고서

Claude와 Codex의 구현·독립 검증 성능, 비용, 소요 시간을 동일 조건에서 비교한 실험 기록

핵심 결론

기본 운영안은 **Codex 구현 → 객관 검사 → 새 Codex 세션의 독립 검증**이다. Claude가 구현한 변경도 Codex가 최종 검증하며, DB·협업·복구 기능은 실제 통합 테스트가 없으면 배포 가능 판정을 내리지 않는다.

실험일 2026년 9월 15일 (한국 표준시)

작성일 2026년 9월 16일

실험 목적

Claude와 Codex가 실제 코딩 에이전트 환경에서 구현과 검증을 담당할 때 성능, 비용, 소요 시간이 어떻게 달라지는지 비교한다. 저장소, 기준 커밋, 구현 과제, 완료 조건, 검증 프롬프트와 객관 검사를 동일하게 유지하고 실행 주체만 교차했다.

평가 과제는 자본주의 경제사 타임라인에 기존 노란 메모와 파란 첨삭 메모에 이어 독립적인 세 번째 **보라색 해석 메모**를 추가하는 것이었다. UI뿐 아니라 클라이언트 직렬화, REST 검증, 서버 CRUD, Drizzle 스키마, 협업 diff 병합, 구버전 클라이언트 보호와 자동화 테스트까지 구현 범위에 포함했다. 운영 데이터, 운영 DB, 실제 배포는 실험에서 제외했다.

실험 조건

시스템	역할	주력 모델	추론 강도	이번 실험의 결제 방식
Claude Code	구현자-검증자	claude-fable-5-1	max	Anthropic API
Codex SDK-Desktop 환경	구현자-검증자	gpt-6-astra	ultra	ChatGPT 구독

Claude Code는 각 Claude 단계에서 내부 상태 정리용 Haiku 호출을 한 번씩 추가로 발생시켰다. 세 번을 합쳐 출력 74토큰, 비용 \$0.007657이었다. 실제 코드 구현과 검증 판단은 모두 Fable 5.1이 수행했다. 두 시스템 모두 내부 서버 에이전트를 비활성화했다.

두 구현은 동일한 **master** 커밋에서 시작했다. 각 구현 결과를 한 번씩 만든 다음 Claude와 Codex가 같은 후보를 서로 독립적으로 검증했다. 따라서 실제 실험에서는 구현 비용을 두 검증 조합이 공유했다. 뒤의 조합별 비용 표에는 각 조합을 처음부터 따로 실행할 경우의 예상 비용을 표시했다.

객관 검사 결과

구현 후보	커밋	타입 검사	단위 테스트	빌드	실제 DB 왕복
Codex 구현	b13345c3fa244788b8ab06fb48c99cff6207bb93	통과	통과, 110개	통과	판정 오류: 격리된 PostgreSQL URL 없음
Claude 구현	f1f5ac6594e3e8849599f071240a7d8c8e7479fd	통과	통과, 107개	통과	판정 오류: 격리된 PostgreSQL URL 없음

일반 테스트는 두 후보 모두 통과했지만, 구버전 초안 복구 경로와 기존 메모의 모든 UI 조작까지 증명하지는 못했다. 실제 검증에서 발견된 중요 결함은 이 사각지대에서 발생했다.

구현 결과 최종 판정

유효한 결함

- P1, 두 후보 공통:** 구버전 초안을 복구하면 저장된 보라색 메모가 삭제될 수 있다. 버전 2 화면은 **refPurple**을 직렬화하지 않는다. 이 화면의 저장 요청은 거절되지만 로컬 초안은 남는다. 최신 버전으로 새로고침한 뒤 초안을 복구하면, 누락된 필드를 버전 3의 명시적인 **기존 값 → null** 삭제 요청으로 해석한다. 서버는 이 요청을 정상 요청으로 받아 병합한다. 실제 직렬화기, 복구 엔진, 서버 검증기와 병합 함수를 사용해 데이터 손실을 재현했다.
- P2, Claude 후보만 해당:** 도구 버튼으로 기존 보라색 메모의 편집기를 열 수 없다. 버튼은 **autoEdit** 값을 변경하지만 **MemoCard**의 편집 상태는 최초 마운트 때만 이 값을 읽는다. 이후 값 변경을 반영하지 않기 때문에 버튼을 눌러도 편집기가 열리지 않았다. Chrome에서 두 번 재현했다. 메모 본문을 직접 클릭하면 편집기는 열린다.
- 낮음, Codex 후보만 해당:** 격리 UI 테스트가 최초 실행에서 간헐적으로 실패한다. Vite의 의존성 최적화가 첫 실행 중 페이지를 다시 불러오면서 테스트 전역 객체가 사라졌다. 다시 실행하면 네 가지 화면 조합이 모두 통과했다. 제품 결함보다는 테스트 인프라의 불안정성에 해당한다.

DB 마이그레이션을 서버 배포보다 먼저 실행해야 한다는 점은 실제 운영 전제 조건이다. 다만 배포가 과제 범위에서 제외되었고 두 후보 모두 DDL과 관련 문서를 만들었으므로 구현 결함으로 계산하지 않았다. 색상 메모가 내보내기 및 무결성 검사에서 빠지는 기존 문제도 이번 기능의 구현 결함으로 계산하지 않았다.

구현 후보 비교

구현자	Claude 검증 결과	Codex 검증 결과	최종 판정
Claude	통과 판정, P1-P2 누락	실패 판정, P1-P2 재현	P1과 P2 존재. 두 후보 중 상대적으로 낮은 품질
Codex	판정 보류, P1 누락	실패 판정, P1 재현	P1과 낮은 수준의 테스트 불안정성 존재. 상대적으로 우수하지만 배포 불가

이번 과제에서는 Codex 구현이 두 후보 중 더 나았다. 하지만 저장된 메모를 삭제할 수 있는 심각한 경로가 남아 있으므로 Codex 후보 역시 수정 없이 배포하면 안 된다.

검증자 성능 비교

최종 판정에서 인정한 중요 결함 사례는 세 건이다. 두 후보에 각각 존재하는 공통 P1 두 건과 Claude 후보에만 존재하는 P2 한 건이다.

Codex 검증자는 세 건을 모두 찾아 직접 재현했다. Claude 검증자는 세 건 모두 찾지 못했고, Claude 후보에는 통과, Codex 후보에는 판정 보류를 내렸다. Claude도 낮은 우선순위의 운영·테스트 공백을 일부 찾았지만 핵심 회귀 문제를 찾아내지는 못했다.

이번 과제에서 가장 큰 추가 가치를 만든 조합은 **Claude 구현을 Codex가 검증한 경우**였다. 잘못된 통과 판정을 뒤집고 두 가지 중요 결함을 모두 찾아냈다. Codex 자체 검증도 공통 P1을 찾았다. Claude 검증은 두 후보의 배포 판단을 실질적으로 개선하지 못했다.

다만 이번 결과는 의도적으로 까다롭게 만든 하나의 합성 과제에서 나온 값이다. 이 프로젝트와 이번 작업 방식에서 관찰한 결과로 보아야 하며, 모델 전체의 일반적인 우열을 확정하는 자료로 사용할 수는 없다.

실제 사용량과 비용

단계	소요 시간	입력 토큰	캐시 입력	캐시 생성	출력 토큰	실제 추가 결제	API 환산 비용
Codex 구현	20분 51초	2,118,011	1,984,768	0	27,717	\$0.000000	\$4.703048
Codex 구현을 Claude가 검증	10분 58초	2,843	961,433	170,227	47,011	\$4.723560	\$4.723560
Codex 구현을 Codex가 검증	6분 50초	1,204,807	1,111,680	0	8,000	\$0.000000	\$2.442950
Claude 구현	13분 40초	2,770	2,051,134	239,526	70,600	\$7.042779	\$7.042779
Claude 구현을 Claude가 검증	13분 08초	2,977	2,281,541	230,674	58,770	\$6.398563	\$6.398563
Claude 구현을 Codex가 검증	11분 03초	1,551,406	1,397,888	0	13,225	\$0.000000	\$3.594318

토큰 항목은 각 공급자의 원래 사용량 형식을 따른다. OpenAI 입력 토큰에는 캐시 입력이 포함된다. Anthropic은 일반 입력, 캐시 읽기와 캐시 생성을 각각 따로 기록한다.

두 구현 후보를 검증자들이 공유한 실제 실험 합계는 다음과 같다.

- Anthropic 실제 결제액: **\$18.164902**
- Codex 실제 추가 결제액: **\$0** — ChatGPT 구독 인증 사용
- Codex API 환산 금액: **\$10.740316**
- 전체 API 환산 금액: **\$28.905218**
- 에이전트 단계 실행 시간 합계: **76분 31초**
- 초기화와 객관 검사를 포함한 전체 소요 시간: 약 **78분 28초**

사전 점검과 무효 실행의 사용량은 위 합계에서 제외했다. 기록이 남은 제외 대상에는 Anthropic 호출이 없었다. Codex 사전 점검과 무효 호출은 약 **\$0.883944**의 API 환산 구독 사용량을 추가로 발생시켰지만 실제 추가 결제는 없었다. 중단된 실행 한 건은 사용량 기록을 만들지 못했으므로, 중단 전 공급자 쪽에서 발생했을 수 있는 미기록 사용량은 하네스로 측정할 수 없다.

각 조합을 독립 실행할 경우의 비용과 시간

구현자 → 검증자	API 환산 비용	순차 실행 시간	검증 결과
Claude → Claude	\$13.441342	26분 48초	잘못된 통과 판정
Claude → Codex	\$10.637097	24분 43초	올바른 실패 판정, P1-P2 발견
Codex → Claude	\$9.426608	31분 48초	판정 오류, P1 누락
Codex → Codex	\$7.145998	27분 41초	올바른 실패 판정, P1 발견

실제 실험에서는 하나의 구현 결과를 두 검증자가 공유했다. 따라서 위 네 행의 비용을 단순 합산하면 Claude와 Codex의 구현 비용이 각각 두 번 계산된다. 이번 실험에서 실제로 관찰한 전체 환산 비용은 **\$28.905218**이다.

다음 실험 권장안

다음 묶음 실험에서는 Codex를 기본 적대적 검증자로 사용한다. 구현자는 Claude와 Codex를 모두 유지해 여러 과제에서 구현 품질과 비용을 계속 비교한다.

다음 실행 전 객관 검사에 아래 두 회귀 시나리오를 추가해야 한다.

1. 구버전 화면에 남은 초안을 최신 화면에서 복구해도 새 필드가 삭제되지 않는지 검사한다.
2. 이미 저장된 메모를 노트 도구 버튼으로 다시 편집할 수 있는지 실제 브라우저에서 검사한다.

격리된 PostgreSQL 테스트 환경도 마련해야 한다. 그래야 실제 저장·재조회, 두 창이 동시 트랜잭션, DDL 역등성, 백업·복원을 최종 판정에 포함할 수 있다.

두 후보 커밋 모두 제품 코드에 반영하지 않는다. 공통 초안 복구 결함을 먼저 수정하고 다시 검증하거나, 현재 제품 브랜치에서 새로운 평가 과제를 시작하는 것이 안전하다.

결론 및 운영 추천안

이번 한 건의 결과만 놓고 보면 **Codex가 구현하고 Codex가 독립 검증하는 흐름**을 기본값으로 삼는 것이 가장 합리적이다. 네 조합 중 API 환산 비용이 가장 낮았고, 구현 품질도 상대적으로 나았으며, 자체 검증에서 실제 데이터 손실 결함을 찾아냈다. 다만 같은 모델의 자기 검증이 언제나 충분하다는 뜻은 아니다. 이번 결과를 기본 운영안으로 사용하면서 과제를 누적해 다시 판단해야 한다.

Claude는 구현 대안이 필요한 과제에 선택적으로 투입하는 편이 좋다. 복잡한 UI 설계나 서로 다른 구현 접근을 비교할 가치가 있을 때 Claude 구현 후보를 추가로 만들고, 최종 검증은 Codex가 맡는다. 이번 실험에서 **Claude → Codex가 Claude 후보의 잘못된 통과 판정을 바로잡았기** 때문에, Claude가 구현한 변경을 Claude 자체 검증만으로 배포하는 흐름은 당분간 피하는 것이 좋다.

실제 프로젝트에는 작업 위험도에 따라 다음과 같이 적용하는 것을 권한다.

- 1. 낮은 위험 작업:** 한 모델이 구현하고 타입 검사·단위 테스트·빌드 같은 객관 검사만 수행한다. 문구, 스타일, 단순 배치처럼 저장 데이터와 무관한 변경이 여기에 해당한다.
- 2. 중간 위험 작업:** 구현과 별도의 새 세션에서 Codex 검증을 필수로 실행한다. 기존 데이터 구조를 읽고 쓰거나 여러 화면에 영향을 주는 기능이 여기에 해당한다.
- 3. 높은 위험 작업:** Codex 독립 검증에 실제 DB·브라우저 통합 테스트를 함께 실행한다. 협업 저장, 동시 편집, 마이그레이션, 복구, 백업·복원처럼 데이터 손실 가능성이 있는 변경이 여기에 해당한다. 필요하다면 Claude 구현 후보를 추가해 두 접근을 비교하되, 두 후보를 모두 배포하지 않고 검증을 통과한 한쪽만 선택한다.

검증 프롬프트에는 “기능이 보이는가”보다 **저장 후 재조회되는가, 구버전과 최신 버전이 섞여도 보존되는가, 두 창이 동시에 수정해도 다른 필드가 사라지지 않는가**를 완료 조건으로 명시해야 한다. 에이전트가 검증했다고 보고하는 것만으로 통과시키지 말고, 각 조건에 대응하는 실행 명령, 브라우저 재현 또는 DB 조회 결과를 증거로 남겨야 한다.

비용 측면에서는 현재 구독 환경에서 Codex의 실제 추가 결제가 없으므로 Codex 독립 검증을 생략할 이유가 적다. Anthropic API는 구현과 검증을 모두 Fable 5.1로 실행하면 비용이 빠르게 증가하므로, Claude는 모든 작업의 상시 검증자보다 대체 구현자 또는 특정 고난도 과제의 보조 검토자로 사용하는 편이 효율적이다.

최종 운영 결론은 다음과 같다.

- 기본 파이프라인은 **Codex 구현 → 객관 검사 → 새 Codex 세션의 독립 검증**으로 둔다.
- Claude가 구현한 작업도 최종 검증은 Codex가 맡는다.
- DB·협업·복구 기능은 실제 PostgreSQL과 브라우저를 연결한 통합 테스트가 없으면 배포 가능 판정을 내리지 않는다.
- 다음 5~10개의 서로 다른 과제에서 같은 방식으로 품질·시간·비용을 누적한 뒤 모델 역할을 다시 조정한다.
- 이번에 발견한 두 회귀 시나리오는 이후 모든 메모·협업 기능의 고정 테스트로 편입한다.

이 방식은 에이전트 숫자를 늘리는 것보다 검증 역할을 분리하고, 실제 데이터 경로를 객관적으로 재현하는 데 초점을 둔다. 이번 파란 메모 사례처럼 화면에서 기능이 만들어지는 것과 저장·복구까지 안전하게 동작하는 것은 별개의 완료 조건이기 때문이다.